

로봇 이동 (삼성 B형 복원)

△ 본 문제는 시험 후 기억을 바탕으로 복원한 연습 문제입니다. 실제 기출 문제와 일부 세부 조건 및 입출력 형식이 다를 수 있습니다.

제한 시간 : 25개 테스트케이스를 합쳐서 Python의 경우 8초

메모리 : 표준

제출 : Main Code 고정 + User Code(init / build / move) 구현

※ 본 문제는 시험 후 기억을 바탕으로 복원한 연습 문제입니다. 실제 기출 문제와 일부 세부 조건 및 입출력 형식이 다를 수 있습니다.

크기가 $N \times N$ 인 격자 형태의 도시가 있다.

각 격자의 좌표는 (x, y) 로 표현하며, x 는 왼쪽에서 오른쪽으로, y 는 위에서 아래로 증가한다.

처음 도시는 비어 있으며, 이후 `build()` 함수가 호출될 때마다 새로운 건물이 추가된다.

건물은 직사각형이며, 좌측 상단 좌표 (mX, mY) , 너비 mW , 높이 mH 가 주어진다.

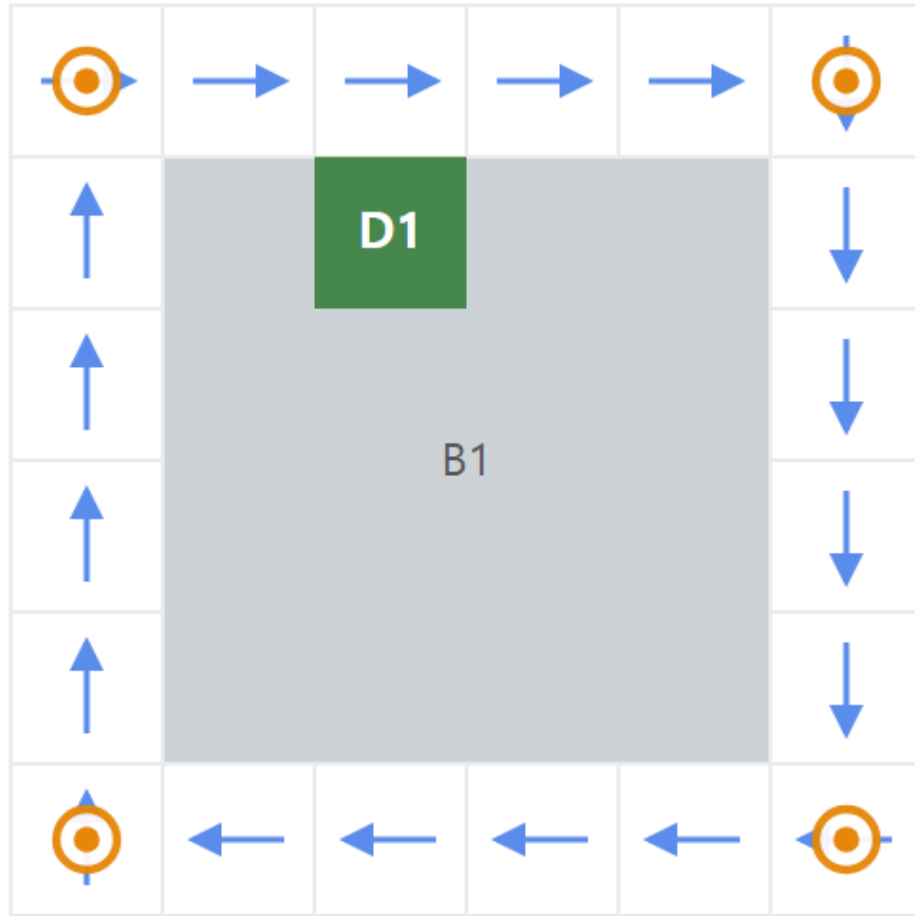
건물이 차지하는 영역은 다음과 같다.

```
mX <= x < mX + mW
mY <= y < mY + mH
```

건물이 추가되면 건물의 외곽을 한 칸 두께로 둘러싸는 도로가 함께 생성된다.

즉 건물의 왼쪽, 오른쪽, 위쪽, 아래쪽으로 한 칸 떨어진 위치에 건물을 둘러싸는 사각형 형태의 도로가 만들어진다.

[Fig. 1] 건물 하나와 그 둘레 도로



→ 시계방향 도로(방향 고정)

◎ 교차로 — 여기서만 다른 도로

건물끼리는 서로 겹치거나 맞닿지 않는다.

단, 서로 다른 건물에 의해 만들어진 도로는 같은 칸을 공유할 수 있으며, 여러 칸이 연속으로 겹칠 수도 있다.

도로의 이동 방향

각 건물을 둘러싸는 도로는 해당 건물을 기준으로 항상 시계 방향으로 이동해야 한다.

따라서 한 건물의 도로에서 이동 가능한 방향은 다음과 같다.

위쪽 도로 : 오른쪽으로 이동
오른쪽 도로 : 아래쪽으로 이동
아래쪽 도로 : 왼쪽으로 이동
왼쪽 도로 : 위쪽으로 이동

도로의 인접한 한 칸으로 이동할 때마다 이동 거리는 1 증가한다.

일반적인 도로에서는 현재 진행 방향을 임의로 변경할 수 없다.

서로 다른 건물의 도로가 같은 칸을 공유하더라도, 일반 도로에서는 다른 건물의 도로로 변경할 수 없다.

교차로

각 건물을 둘러싸는 사각형 도로의 네 꼭짓점은 교차로이다.

서로 다른 건물의 도로가 교차로에서 연결될 수 있으며, 하나의 교차로에 여러 건물의 도로가 연결될 수도 있다.

로봇이 교차로에 도착하면 현재 진행하던 도로를 계속 따라갈 수도 있고, 다른 건물의 도로로 이동할 수도 있다.

교차로에서는 직진, 좌회전, 우회전, U턴이 모두 가능할 수 있다.

단, 이동하려는 방향에 실제 도로가 존재해야 하며, 이동한 이후에도 어떤 건물을 기준으로 한 시계 방향 이동 조건을 만족해야 한다.

즉 서로 다른 건물의 도로로 변경하는 것은 교차로에서만 가능하다.

교차로에서 다른 방향을 선택하는 행위 자체에는 별도의 이동 비용이 없으며, 인접한 도로 한 칸으로 실제 이동할 때 거리 1이 증가한다.

출입구

각 건물에는 출입구가 정확히 하나 존재한다.

출입구의 위치는 건물의 좌측 상단을 기준으로 한 상대 좌표 ($mDoorX$, $mDoorY$)로 주어지므로, 출입구의 실제 좌표는 ($mX + mDoorX$, $mY + mDoorY$)이다.

출입구는 반드시 건물의 가장자리 칸에 존재하며, 건물 바깥의 도로 한 칸과 인접한다.

로봇은 출입구를 드나들 때 항상 우회전해야 한다.

즉 로봇이 도로를 진행하고 있을 때, 현재 진행 방향의 오른쪽 한 칸에 어떤 건물의 출입구가 존재하는 경우에만 해당 건물로 들어갈 수 있다.

도로에서 출입구로 이동하거나 출입구에서 도로로 이동할 때 각각 이동 거리 1이 증가한다.

건물 내부에서는 별도의 이동 거리가 발생하지 않는다.

출입구와 인접한 도로는 교차로가 아니며, 이후 새로운 건물이 추가되더라도 이 조건은 유지된다.

구현할 함수

사용자는 다음 세 함수를 구현해야 한다.

[1] `init(N)`

각 테스트 케이스의 시작에 한 번 호출된다. 크기 $N \times N$ 의 빈 도시를 초기화한다.
 N : 도시의 한 변의 길이

[2] `build(mID, mX, mY, mW, mH, mDoorX, mDoorY)`

새로운 건물을 추가한다.
 mID : 건물의 고유 ID
 mX , mY : 건물 좌측 상단의 좌표
 mW , mH : 건물의 너비와 높이
 $mDoorX$, $mDoorY$: 건물 좌측 상단을 기준으로 한 출입구의 상대 좌표
같은 mID 를 가진 건물이 중복해서 추가되는 경우는 없다.

[3] `move(mStart, mEnd, M, mID)`

건물 mStart의 출입구에서 출발하여 건물 mEnd의 출입구까지 이동한다.
 이때 mID[0]부터 mID[M-1]까지 주어진 모든 건물의 출입구를 반드시 방문해야 한다.
 경유 건물을 방문하는 순서는 자유이며, 모든 조건을 만족하며 이동했을 때의 최소 이동 거리를 반환한다.
 mStart : 출발 건물 ID
 mEnd : 도착 건물 ID
 M : 반드시 방문해야 하는 경유 건물의 개수 (최대 5)
 mID : 경유 건물의 ID 배열
 이동이 불가능한 경우는 주어지지 않는다.

예제

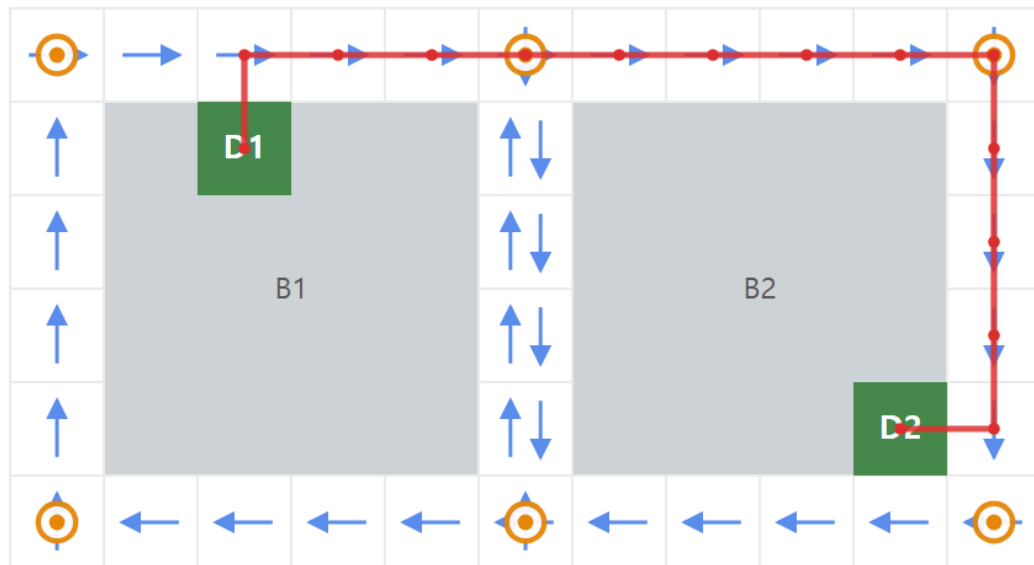
다음과 같이 두 건물이 존재한다고 하자.

건물 1의 출입구에서 출발한 로봇은 건물 1의 도로에 진입한 뒤 시계 방향으로 이동한다.

두 건물의 도로가 연결된 교차로에 도착하면 건물 2의 시계 방향 도로로 이동할 수 있으며,

이후 건물 2의 출입구가 진행 방향의 오른쪽에 위치했을 때 우회전하여 건물 안으로 들어갈 수 있다.

[Fig. 2] D1 -> D2 최단경로 (빨간 선, 14칸)



→ 시계방향 도로(방향 고정)

⊙ 교차로 — 여기서만 다른 도로로 갈아탈 수 있다

도로가 단방향이므로 일반적으로 두 건물 사이의 거리는 대칭이 아니다.

즉 $\text{distance}(A, B) \neq \text{distance}(B, A)$ 일 수 있다.

순서	함수	반환값
1	init(15)	
2	build(1, 3, 3, 4, 4, 1, 0)	
3	build(2, 8, 3, 4, 4, 3, 3)	
4	move(1, 2, 0, {})	14
5	move(2, 1, 0, {})	20
6	move(1, 1, 0, {})	0

순서	함수	반환값
1	init(24)	
2	build(1, 15, 12, 3, 5, 1, 4)	
3	build(2, 9, 9, 5, 6, 1, 0)	
4	move(2, 1, 0, {})	21
5	move(2, 1, 0, {})	21
6	build(3, 12, 5, 2, 3, 1, 1)	
7	move(2, 3, 0, {})	12
8	move(3, 2, 1, {1})	40
9	build(4, 6, 16, 3, 6, 0, 4)	
10	build(5, 10, 16, 2, 6, 0, 5)	
11	build(6, 15, 5, 5, 6, 4, 1)	
12	move(1, 5, 0, {})	17
13	build(7, 3, 6, 5, 6, 4, 3)	
14	move(4, 6, 0, {})	35
15	move(3, 6, 0, {})	16
16	move(5, 7, 2, {3, 1})	69
17	move(6, 5, 4, {2, 3, 1, 7})	82
18	move(7, 2, 0, {})	11
19	move(6, 7, 1, {2})	59
20	move(7, 3, 5, {4, 2, 6, 1, 5})	101
21	move(7, 1, 4, {2, 5, 4, 3})	92
22	move(4, 5, 4, {2, 1, 6, 3})	81
23	move(7, 6, 1, {3})	37
24	move(1, 4, 4, {7, 5, 2, 6})	86
25	move(6, 5, 2, {2, 3})	66

첫 번째 테스트 케이스

아래는 제공되는 Sample Input 의 첫 번째 테스트 케이스이다.

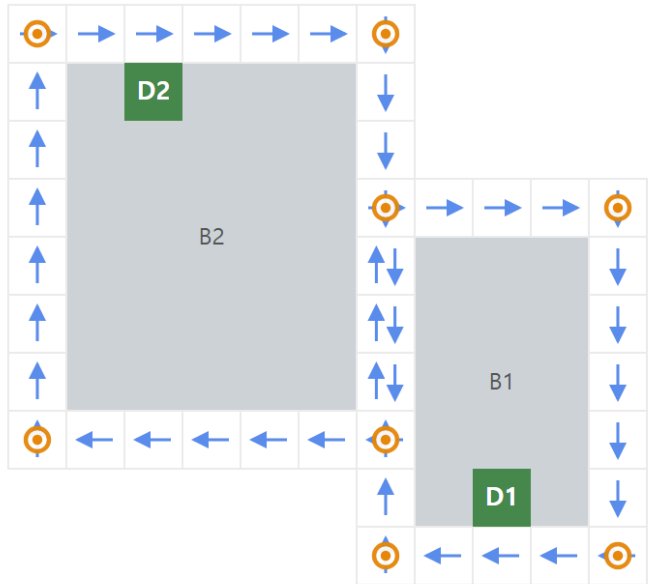
한 테스트 케이스는 init 1회와 build/move 명령이 섞여 총 25개의 명령으로 구성된다.

(첫 케이스는 건물 7개로 작지만, 뒤쪽 케이스에는 건물이 1,000개까지 들어간다.)

도시가 만들어지는 과정

건물이 하나씩 늘어나면서 도로가 이어지고 교차로가 생긴다.

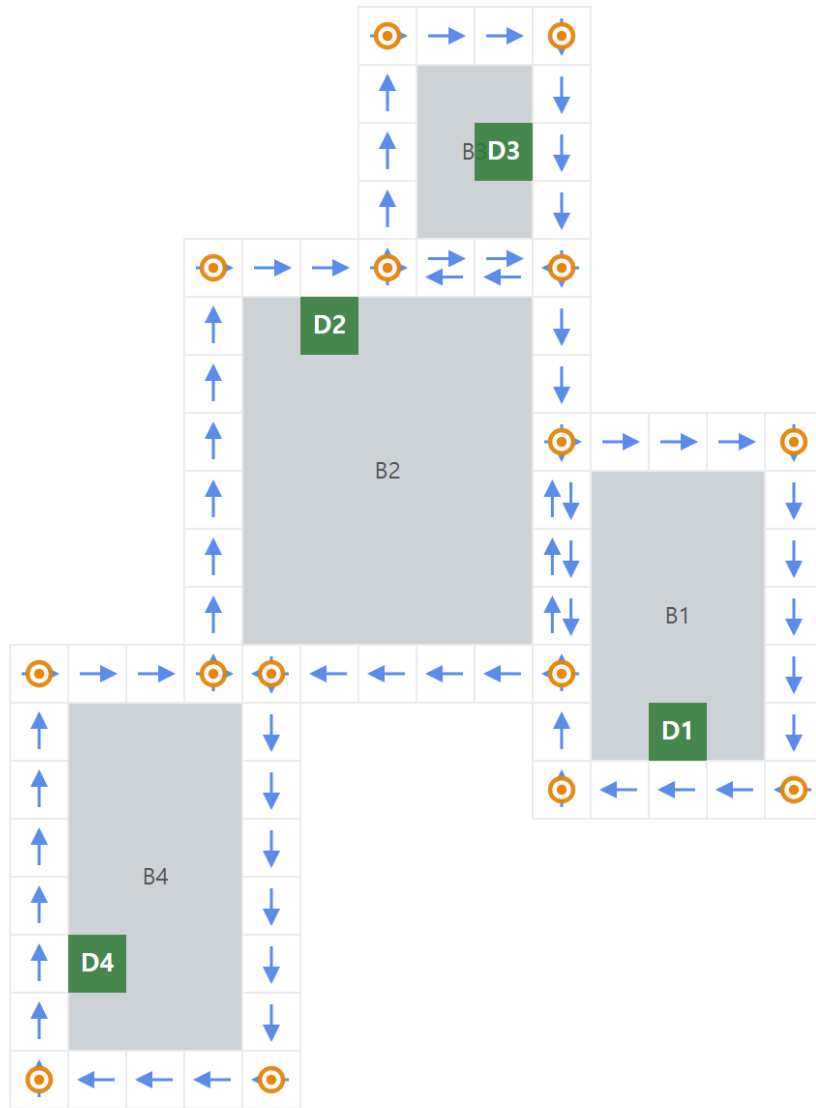
[Fig. 3] 건물 2개를 지은 뒤



→ 시계방향 도로(방향 고정)

⊙ 교차로 — 여기서만 다른 도로로 갈아탈 수 있다

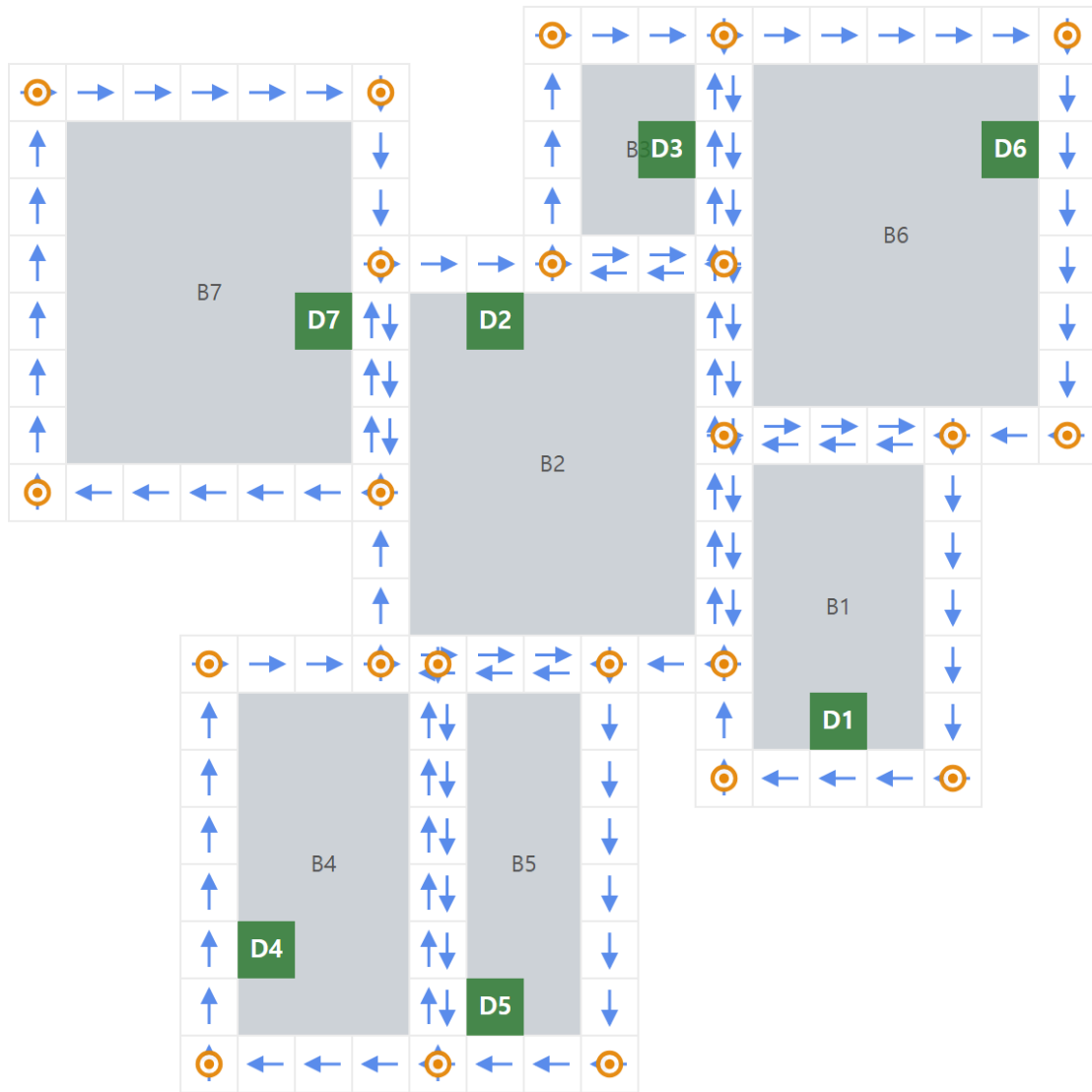
[Fig. 4] 건물 4개를 지은 뒤



→ 시계방향 도로(방향 고정)

⊙ 교차로 — 여기서만 다른 도로로 갈아탈 수 있다

[Fig. 5] 건물 7개를 지은 뒤



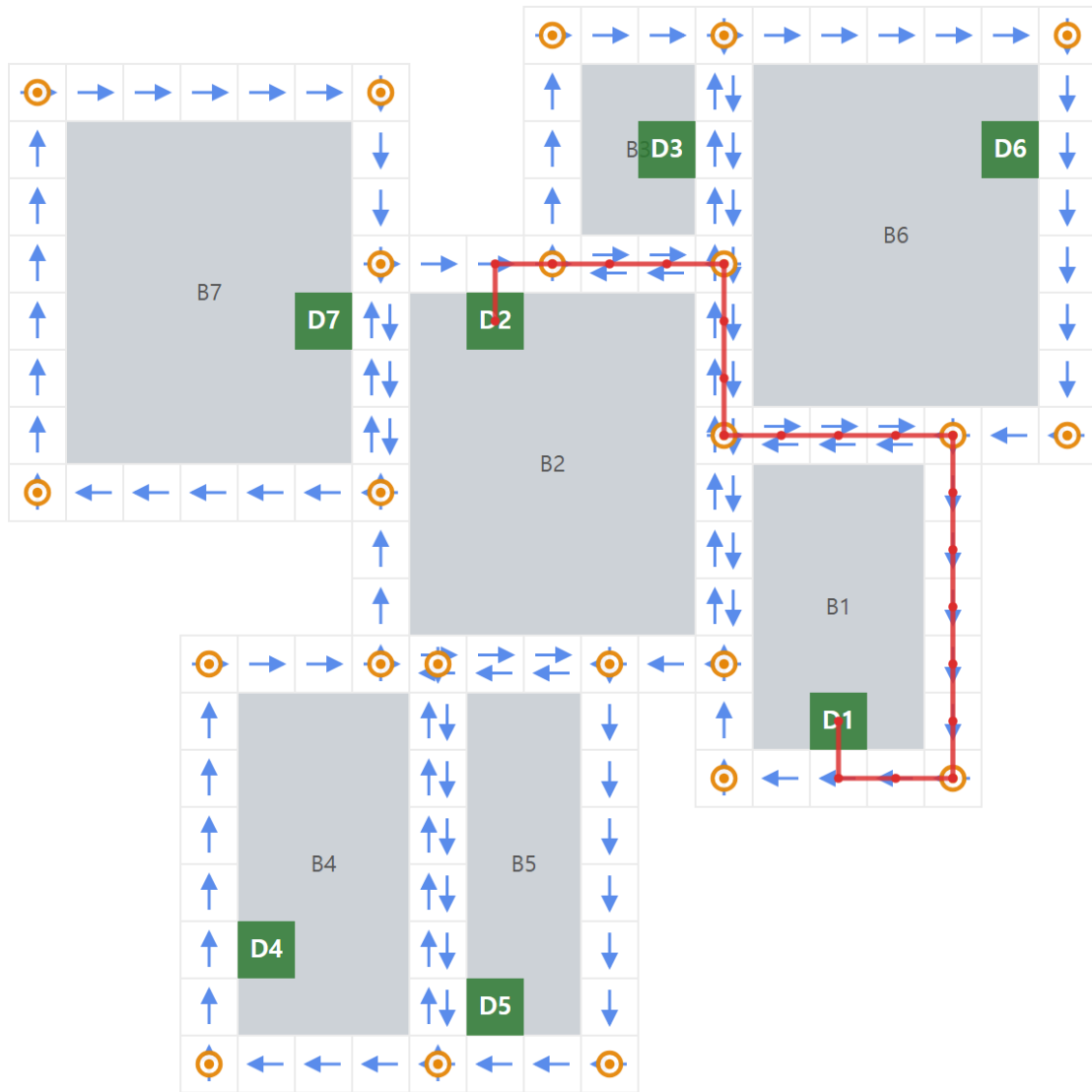
→ 시계방향 도로(방향 고정)

⊙ 교차로 — 여기서만 다른 도로로 갈아탈 수 있다

도로가 겹친 칸에는 화살표가 나란히 두 개 그려져 있다. 이는 서로 다른 건물의 도로가 겹쳐 있다는 뜻이지 그 자리에서 방향을 바꿀 수 있다는 뜻이 아니다.
 방향을 바꿀 수 있는 곳은 주황색 원으로 표시된 교차로뿐이다.

move 의 최단경로

[Fig. 6] 명령 4: `move(2, 1, 0, {})` = 21



→ 시계방향 도로(방향 고정)

⊙ 교차로 — 여기서만 다른 도로로 갈아탈 수 있다

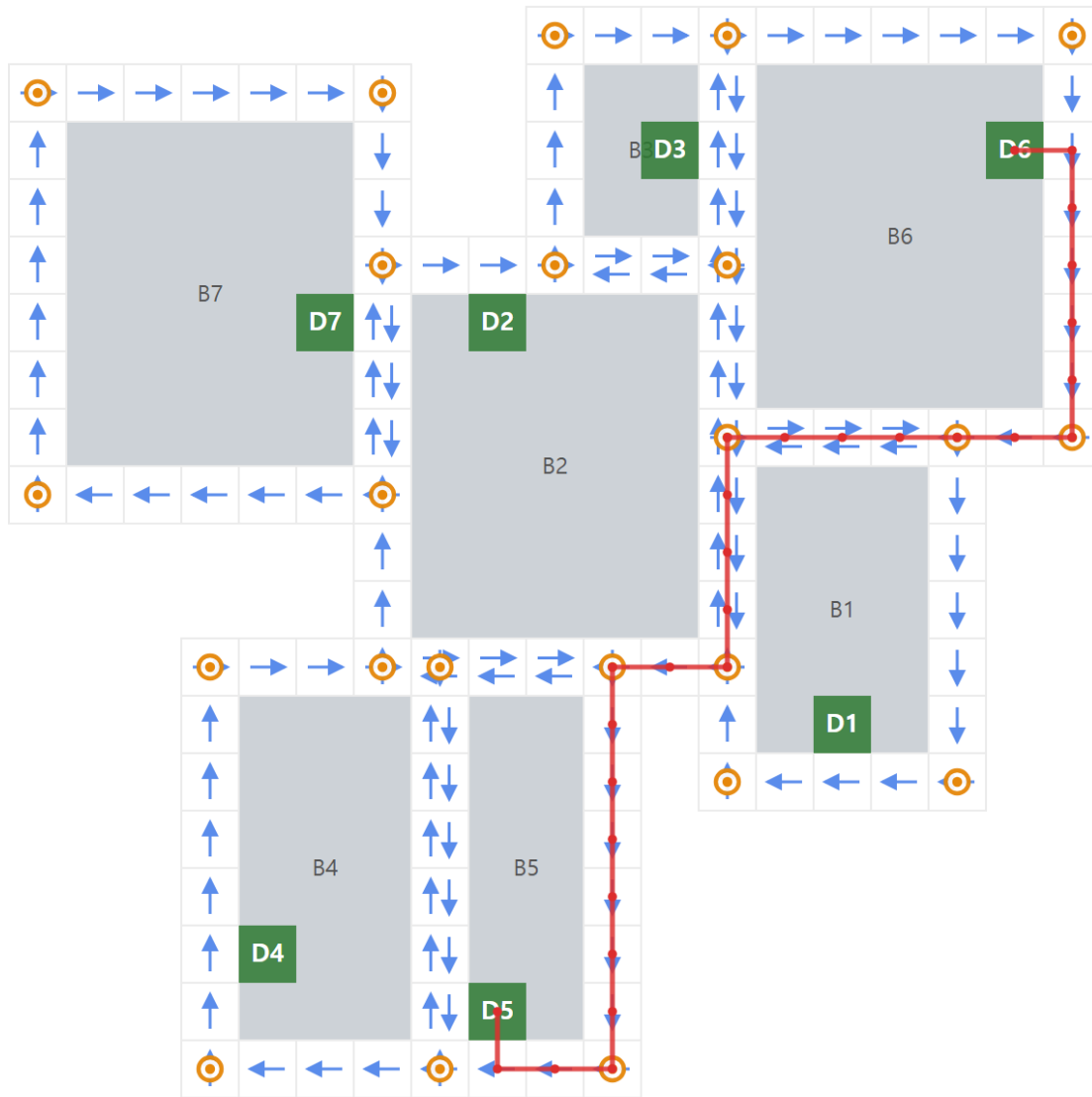
[Fig. 7] 명령 8: move(3, 2, 1, {1}) = 40 (그림은 경유 없는 3→2 최단경로 26)

→ 시계방향 도로(방향 고정)

◎ 교차로 — 여기서만 다른 도로로 갈아탈 수 있다

◎ 교차로 — 여기서만 다른 도로로 갈아탈 수 있다

[Fig. 8] 명령 17: `move(6, 5, 4, {2, 3, 1, 7}) = 82` (그림은 경유 없는 6→5 최단경로 28)



→ 시계방향 도로(방향 고정)

⊙ 교차로 — 여기서만 다른 도로로 갈아탈 수 있다

출발 건물의 출입구에서 나와 시계 방향으로 이동하다가, 교차로에서 다른 건물의 도로로

갈아탄 뒤, 진행 방향의 오른쪽에 목적지 출입구가 나타났을 때 우회전하여 들어간다.

경유지가 있는 명령은 경유 순서를 모두 따져 가장 짧은 합을 구해야 한다.

입출력 형식

입출력은 제공되는 Main Code에서 처리하며 User Code에서는 별도의 입출력을 구현하지 않는다.

각 명령은 다음과 같다.

```
100 N
    init(N) 을 호출한다.

200 mID mX mY mW mH mDoorX mDoorY
    build(mID, mX, mY, mW, mH, mDoorX, mDoorY) 를 호출한다.

300 mStart mEnd M mID[0] ... mID[M-1] expected
    move(mStart, mEnd, M, mID) 를 호출한다.
    반환값이 expected 와 다르면 해당 테스트 케이스는 오답으로 처리된다.
```

각 테스트 케이스의 모든 `move()` 결과가 정답과 일치하면 그 테스트 케이스의 점수를 획득한다.

제약 사항

- 각 테스트 케이스의 시작에는 `init()` 이 한 번 호출된다.
- 건물의 ID는 서로 다르다.
- 건물끼리는 서로 겹치거나 맞닿지 않는다.
- 서로 다른 건물의 도로는 겹칠 수 있다.
- 경유 건물의 수 M 은 최대 5이다.
- 도로는 방향성을 가지므로 출발지와 도착지를 뒤바꾸면 최단 거리가 달라질 수 있다.
- `move()` 에서 조건을 만족하는 경로가 존재하지 않는 경우는 주어지지 않는다.
- `build()` 는 한 테스트 케이스에서 최대 약 1,000회 호출되는 것으로 복원하였다.
- `move()` 는 한 테스트 케이스에서 최대 약 100회 호출되는 것으로 복원하였다.